EXPERIMENT 13.1.1

13.1.1 Fibonacci sequence using modules

Algorithm:

1.Start

2. Define a function generate_fibonacci_sequence(max_value)

3.Initialize an empty list fib_sequence

4. Initialize first two Fibonacci numbers:

a = 0

b = 1

5.While a <= max_value, repeat:

Add a to fib_sequence

Compute next term:

- next = a + b
- Update a = b
- Update b = next

6. Return fib_sequence

7. stop

Code:

Main.py

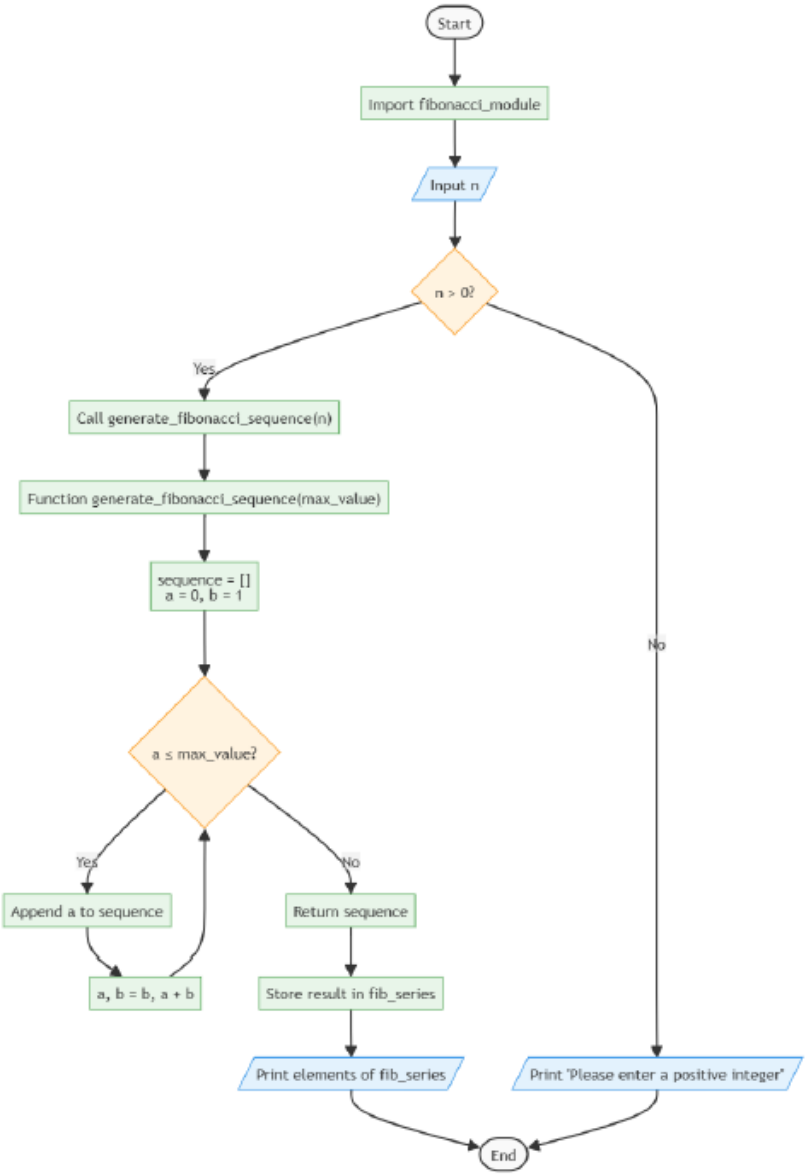
```
import fibonacci_module
n = int(input())

if n > 0:
fib_series = fibonacci_module.generate_fibonacci_sequence(n)
print(' '.join(map(str, fib_series)))
else:
print("Please enter a positive integer")
```

Fibonacci.py

```
def generate_fibonacci_sequence(max_value):
sequence = []
a, b = 0, 1
while a <= max_value:
sequence.append(a)
a, b = b, a + b
return sequence
```

Flowchart



anwesh.lanjewar.batch2025@sitnagpur.siu.edu.in Support Logout

13.1.1. Fibonacci Sequence using Modules

13:43

Write a Python program using modular programming to generate the Fibonacci sequence up to a given maximum value.

Module Requirements

- Create a module **fibonacci_module.py** with a function **generate_fibonacci_sequence(max_value)** that returns list of all Fibonacci numbers less than or equal to the maximum value.

Main Program Requirements

Create **main.py** that performs the following:

- Import the **fibonacci_module.**
- Read an integer n from user input.
- Validate if n is positive.
- If valid, call the module function and display the Fibonacci sequence.
- If invalid, display an error message.

Input Format:

- Single integer n representing the maximum value for Fibonacci sequence.

Output Format:

- If $n > 0$, print the Fibonacci sequence as space-separated numbers.
- If $n \leq 0$, print: "Please enter a positive integer".

Note:

- Two files are provided: **fibonacci_module.py** and **main.py**. The **main.py** file handles input and output. You need to complete both files based on the question requirements.

Sample Test Cases +

Explorer

main.py | fibonacci_...

Submit

```python
# Write your code here......
import fibonacci_module
n = int(input())

if n > 0:
    fib_series = fibonacci_module.generate_fibonacci_sequence(n)
    print(' '.join(map(str, fib_series)))
else:
    print("Please enter a positive integer")
```

Debugger

Average time 0.004 s 4.25 ms

Maximum time 0.007 s 7.00 ms

4 out of 4 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 7 ms Debug

Expected output	Actual output
10	10
0 1 1 2 3 5 8	0 1 1 2 3 5 8

Test case 2 4 ms

Test case 3 3 ms

Terminal Test cases

< Prev Reset Submit Next >

anwesh.lanjewar.batch2025@sitnagpur.siu.edu.in Support Logout

13.1.1. Fibonacci Sequence using Modules

13:48

Write a Python program using modular programming to generate the Fibonacci sequence up to a given maximum value.

Module Requirements

- Create a module **fibonacci_module.py** with a function **generate_fibonacci_sequence(max_value)** that returns list of all Fibonacci numbers less than or equal to the maximum value.

Main Program Requirements

Create **main.py** that performs the following:

- Import the **fibonacci_module.**
- Read an integer n from user input.
- Validate if n is positive.
- If valid, call the module function and display the Fibonacci sequence.
- If invalid, display an error message.

Input Format:

- Single integer n representing the maximum value for Fibonacci sequence.

Output Format:

- If $n > 0$, print the Fibonacci sequence as space-separated numbers.
- If $n \leq 0$, print: "Please enter a positive integer".

Note:

- Two files are provided: **fibonacci_module.py** and **main.py**. The **main.py** file handles input and output. You need to complete both files based on the question requirements.

Sample Test Cases +

Explorer

main.py | fibonacci...

Submit

```python
# Write your code here.....
def generate_fibonacci_sequence(max_value):
    sequence = []
    a, b = 0, 1
    while a <= max_value:
        sequence.append(a)
        a, b = b, a + b
    return sequence
```

Debugger

Average time 0.004 s 4.25 ms

Maximum time 0.007 s 7.00 ms

4 out of 4 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 7 ms Debug

Expected output	Actual output
10	10
0 1 1 2 3 5 8	0 1 1 2 3 5 8

Test case 2 4 ms

Test case 3 3 ms

Terminal Test cases

< Prev Reset Submit Next >